

CSCE 638 NATURAL LANGUAGE PROCESSING · MODULE 10 · LECTURE NOTES

Evaluation

The hard part, and usually the weakest part.

MODULE OUTCOMES

- Design an evaluation from the decision it supports.
- Choose and justify a baseline.
- Explain why generation metrics mislead.
- Explain benchmark contamination and leakage.
- Run an error analysis that changes decisions.

1 From the decision

START FROM WHAT SOMEONE WILL DO DIFFERENTLY, AND DERIVE THE METRIC FROM THAT

A metric chosen because it is standard measures what was convenient to measure — which may correlate with what you care about, **and the correlation is an empirical claim rather than an assumption you are entitled to make.**

So ask: what decision does this output support? If a person will read the top three results, measure the top three; if a system will act automatically on the single top answer, measure precision at one — and those two systems should be optimised differently.

And the error costs are almost always asymmetric. A false positive and a false negative rarely cost the same thing, so a single accuracy number averages over a distinction that determines whether the system is usable.

Which is CSCE 633 Module 12's choose-the-metric-from-the-use argument — and CSCE 701 Module 09 §1's **design-backwards direction**, applied to measurement rather than to logging. **Three courses, one direction of reasoning.**

- **Look at the test set once. Every look is a decision informed by it** — which architecture, which hyperparameters, which preprocessing — and **after a dozen looks you have fitted the test set with your own judgement** as surely as gradient descent would have.
- **Split by the unit you will generalise over** — by document, by user, or by time, rather than by sentence, if sentences from a single document would otherwise appear on both sides of the split and leak.
- **And split by time if your data is temporal**, because **a random split lets the model see the future** (CSCE 633 Module 11) — which is the commonest serious leak in applied work and makes a system look far better than it will be.

- **Check for near-duplicates across the splits**, which are **extremely common in scraped text** and inflate every score — reposted articles, templated pages, and boilerplate. **This is the one that silently ruins text experiments**, and **an exact-hash check is not enough**; you need near-duplicate detection (CSCE 676).
- **And report a confidence interval**, because **a difference of half a point on a thousand examples is noise** — and a great deal of reported improvement is within the interval that was not computed.

2 Baselines

MAJORITY CLASS

the single most common label. On a skewed task this is embarrassingly strong, and reporting accuracy without it is uninformative.

RANDOM, AND HUMAN AGREEMENT

the floor and a realistic ceiling. If annotators agree with each other only 80% of the time, then 85% accuracy is not obviously improvable.

LEXICAL / RETRIEVAL

TF-IDF or BM25 plus logistic regression. Cheap, and it beats a surprising number of published neural systems (CSCE 670 Module 03).

EMBEDDINGS + LOGISTIC REGRESSION

Module 04 section 4's baseline. One hour of work.

THE PREVIOUS SIMPLEST THING

whatever the system you are replacing already does, measured on the same split

AND A SHORTCUT CHECK: train on a corrupted version of the input (shuffled words, labels only, the question without the passage). If it still scores well, the task has an artefact.

The shortcut check is the most informative single experiment in this module, and it costs almost nothing: **it tells you whether your benchmark measures the ability you named or something else entirely**. **The input-ablation check has invalidated several famous benchmarks** — natural language inference datasets where the hypothesis alone predicted the label, reading comprehension sets answerable without the passage, and visual-question datasets answerable without the image. **Run it on your own task before trusting any number from it.**

A SYSTEM THAT BEATS A STRAWMAN HAS ESTABLISHED NOTHING

The baseline has to be the best cheap alternative, tuned with the same effort you spent on your own system — and an untuned baseline against a carefully tuned system is not a comparison, it is a demonstration of where the effort went.

Which is resisted because a strong baseline sometimes wins — and that is a result worth having, since it saves the entire cost of building, deploying, and maintaining the complicated system. A negative result that saves a quarter of engineering is valuable.

And the replication literature is unambiguous on this point: a substantial share of reported gains disappear when the baselines are tuned with equal effort, a finding reproduced across recommendation, retrieval, and language modelling.

So Project 2 grades the baseline first — because an unbeaten strong baseline is a finding and a beaten weak one is not, and getting that ordering right is most of what distinguishes a useful evaluation from a persuasive one.

3 Generation metrics

Metric	What it measures	What it misses
BLEU	N-gram overlap with one or more references.	Any correct paraphrase, and meaning entirely — a perfect translation using different words scores poorly.
ROUGE	Overlap, recall-oriented, for summarisation.	The same, plus factual correctness — a summary can score well while asserting something false.
BERTScore and similar	Embedding similarity to the references.	Factuality; and it inherits the scoring model's own biases (Module 04 §3).
Perplexity	How well the model fits held-out text.	Usefulness (Module 03 §3).
LLM-as-judge	Another model's stated preference.	Calibration; and it prefers its own style — see the callout.
Human rating	What a particular rater thought.	Consistency, unless the protocol is tight — and inter-rater agreement must be reported.

Every row misses factual correctness, which for most real applications is precisely the thing that matters — and that gap is why generation evaluation remains an open problem rather than a solved one with a preferred metric. The honest consequence is that a generation system's quality claim needs a task-specific factuality check that you build, and that no general metric supplies.

THE JUDGE-MODEL PROBLEM DESERVES NAMING

Using a strong model to score outputs is cheap, fast, and correlates with human judgement considerably better than overlap metrics do — which is why it spread so quickly, and the correlation is real rather than illusory.

And it has specific, measured biases: a preference for longer answers, a preference for its own stylistic and formatting conventions, and a preference for whichever option is presented first — all documented, and all correctable only partially (position bias by randomising, the others not really).

Which means it cannot evaluate a competitor fairly if that competitor writes in a different style — and it certainly cannot evaluate factuality that it does not itself know, which is the failure that matters most and the one least often acknowledged.

So use it for development and for triage, calibrate it against human judgement on a sample, and report that calibration — which almost nobody does, and which is exactly what would make the method trustworthy. The calibration sample can be small; it just has to exist.

4 Error analysis, and contamination

- Read at least twenty errors individually, with the input, the prediction, and the correct answer side by side — not aggregated, not sampled by score, just read.

- **Let the categories come from the data rather than from your expectations — which is the whole point, and is the step people skip by starting from a taxonomy they already had**, thereby learning nothing they did not already believe.
- **Count the categories**, so that you know which is worth fixing — and **the distribution is usually far more skewed than anticipated**, with one or two categories accounting for most of the loss and the rest being a long tail.
- **Check the labels while you are in there**, because **a meaningful fraction of apparent 'errors' are annotation mistakes** in most datasets — and a test set with a 5% label error rate caps your measurable accuracy at 95% no matter what you build.
- **And look at the *confident* errors specifically** — high-probability wrong answers — which **are the dangerous ones in deployment and the most diagnostic in analysis** (Module 12 §3). **Reading twenty errors is a better use of an hour than any hyperparameter sweep, and it is the only method that finds a mislabelled test set.**

AND BENCHMARK CONTAMINATION

A pretrained model may well have seen your test set, because **it was trained on a crawl of the web and your benchmark is published on the web** — frequently along with its answers, in papers, in repositories, and in blog posts.

Which inflates scores in a way that is hard to detect and impossible to rule out without access to the training data — which you generally do not have, and which even the model's authors may not be able to search exhaustively.

So prefer a test set you constructed yourself, from data postdating the training cutoff, or at minimum check for verbatim and near-verbatim overlap and report what you checked — which is a reportable fact rather than a guarantee.

And treat public benchmark scores as weak evidence. The ranking may reflect exposure rather than capability, and **your own held-out data is worth more than a leaderboard position** — which is Module 13 §2's claim form and is the practical conclusion of this module.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Jurafsky & Martin, the evaluation sections (free)	§1 and §3 , with the metric definitions and their known failures.
Gururangan et al. — Annotation Artifacts in NLI Data (free)	§2's shortcut check, applied — the hypothesis-only baseline that reframed a whole benchmark.

Resource	What it covers for this module
Reiter — A Structured Review of the Validity of BLEU (free)	§3's first row, assessed carefully — where BLEU correlates with judgement and where it does not.
Zheng et al. — Judging LLM-as-a-Judge (free)	§3's callout with the measurements — the position, verbosity, and self-preference biases quantified.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write the decision** your Project 2 output supports, then derive the metric from it.
- 2 **State the cost asymmetry** between your two error types.
- 3 **Check your splits for near-duplicates** and report how many you found.
- 4 **Split your data by time** and compare the score against a random split.
- 5 **Compute the majority-class and human-agreement numbers** for your task.
- 6 **Run the shortcut check**: train on corrupted input and report the score.
- 7 **Tune a strong baseline with the same effort** you spent on your system.
- 8 **Score ten generated outputs with BLEU and by reading them**, and compare the rankings.
- 9 **Calibrate a judge model** against your own judgement on twenty examples.
- 10 **Read twenty errors**, derive the categories from them, count them, and report how many were label errors.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why derive the metric from the decision?
- 2 Why is a metric's correlation with what you care about an empirical claim?
- 3 Give five rules about splits, and the one that silently ruins text experiments.
- 4 Name five baselines you must beat.
- 5 What is the shortcut check, and what has it invalidated?
- 6 Why is an unbeaten strong baseline a finding?
- 7 Name six generation metrics and what each misses.
- 8 What do they all miss?
- 9 Give three biases of a judge model and the one correction that matters.

10 Give five rules for error analysis and say which errors are most diagnostic.